# Infrastructure as Python: From AWS CLI and Bash to boto3 and pytest

**Course:** ITMO 444/544 Cloud Computing / Cloud Security
**Level:** Advanced (graduate)
**Estimated Time:** 180 to 210 minutes
**Prerequisites:** The previous lab (`lab04`) completed: S3 bucket, IAM roles and instance profiles, launch templates, ALB, target groups, Auto Scaling Groups, `.env`, and the bash create and delete scripts. You will copy that lab's folder to a new `lab05` folder in Part 0 and work only there. Python 3.9 or newer.

---

## 1. Learning Objectives

By the end of this lab you will be able to:

1. Translate AWS CLI commands into **boto3** calls, and explain how CLI options, `--query` filters, waiters and pagination map to the SDK.
2. Replace `2>/dev/null || true` with **targeted exception handling** that ignores only the errors you expect.
3. Rebuild the full Lab04 stack (S3, IAM, launch templates, ALB, target groups, Auto Scaling Groups) as a small, importable Python package with idempotent `ensure_*` functions.
4. Keep configuration, policy documents and user-data generation in **pure functions** that can be tested without AWS.
5. Write **unit tests** that run the real create and delete code against a fake AWS (`moto`), at no cost and in seconds.
6. Write **integration tests** that check the infrastructure you actually deployed: configuration, least-privilege IAM, network exposure, and end-to-end application behavior.
7. Write a **chaos test** that terminates an instance and asserts recovery within your SLO.
8. Explain what each testing layer can and cannot prove, and why you need both.

---

## 2. Why This Matters

Bash and the AWS CLI are the right tool for exploring AWS and for short scripts. The Lab04 scripts show where they stop being the right tool:

| Pain in the bash version | What Python gives you |
|---|---|
| Output is text. You parse it with `--query ... --output text` and compare against the string `"None"` | Responses are dictionaries and lists. Missing things are `None` or an exception |
| `2>/dev/null \|\| true` hides every failure, including typos and permission errors | `except ClientError` with a specific error code. Anything unexpected still stops the script |
| Uploader and viewer logic is copy-pasted, or hidden behind string-built function arguments | A `Tier` object and a loop. One function, called twice |
| JSON policies are built by heredocs with shell variable expansion | Python dictionaries. Valid JSON by construction, and testable |
| The only way to test is to run it against AWS and look | Pure functions get unit tests. The whole stack gets tested against a fake AWS |
| Waiting is a hand-written `sleep` loop | Built-in **waiters** and paginators |

Nothing here is a criticism of the CLI. The CLI is itself written in Python on top of the same library you will use today (`botocore`), so every command you know has a direct SDK equivalent.

---

## 3. Free Tier and Cost Guide

This lab creates the **same AWS resources as Lab04**. It adds no new services.

| Activity | Cost |
|---|---|
| Unit tests ( `pytest` ) | **Free.** They run against `moto` , an in-memory fake. They need no AWS account and no credentials |
| Building the stack with `create_stack.py` | The same as `create_app_stack.sh` : two `t3.micro` instances and one ALB, billed by the hour while they exist |
| Integration and chaos tests | Run against the real stack, so they cost whatever the stack costs while they run |

> ⚠ An idle ALB still bills hourly. Build the stack, run the integration tests, then run `python delete_stack.py` in the same sitting.

A good working rhythm is to do Parts 0 to G with **no stack running** (unit tests only), and then build the stack once for Parts H and I.

---

## 4. What You Will Build

```
lab05/
├── .env                        (unchanged, the same file the bash scripts read)
├── create_stack.py             (replaces create_app_stack.sh)
├── delete_stack.py             (replaces delete_app_stack.sh)
├── stack/                      (the package that does the work)
│   ├── config.py               settings from .env, and the Tier object
│   ├── outputs.py              ids the create step generates
│   ├── clients.py              the boto3 clients
│   ├── util.py                 error handling helpers
│   ├── policies.py             IAM policy documents
│   ├── storage.py              S3 bucket, IAM roles and instance profiles
│   ├── compute.py              user-data and launch templates
│   ├── network.py              VPC, subnets, security groups
│   ├── balancer.py             target groups, ALB, listeners
│   ├── scaling.py              Auto Scaling Groups
│   └── runner.py               create_stack() and delete_stack()
└── tests/
    ├── conftest.py             shared fixtures
    ├── helpers.py              probe() and wait_until()
    ├── unit/                   fast, free, no AWS
    └── integration/            checks against your deployed stack
```

**Three layers of testing, three different questions:**

| Layer | Command | Question it answers | Needs AWS | Time |
|---|---|---|---|---|
| Unit | `pytest` | Is my code correct? | No | seconds |
| Integration | `pytest -m integration` | Is the deployed infrastructure correct and does the app work? | Yes | a few minutes |
| Chaos | `pytest -m chaos -s` | Does it recover from failure within the SLO? | Yes | several minutes |

---

## 5. Part 0: Set Up Your `lab05` Folder

This lab does not edit the previous lab's folder. You copy it to a new `lab05` folder and do **all** of your work there. Your `lab04` folder stays untouched as a record of what you built before.

### 0.1. Copy the previous lab

From the root of your course repo:

```
cp -r lab04 lab05
cd lab05
ls -a
```

The bash scripts, the `testing/` folder and `stack_outputs.env` come along. Leave them where they are, because you will compare them with the Python version.

> ⚠ Treat bash and Python as two ways to manage **the same stack**, not two stacks. Do not run `create_app_stack.sh` and `create_stack.py` back to back without deleting in between. Both find resources by the names in `.env`, so each one can delete what the other built.

### 0.2. Create a virtual environment and install the tools

```
python3 -m venv .venv
source .venv/bin/activate
```

Create `requirements.txt`:

```
boto3>=1.34
requests>=2.31
pytest>=8.0
moto[ec2,elbv2,autoscaling,iam,s3]>=5.0
```

```
pip install -r requirements.txt
printf '.venv/\n__pycache__/\n.pytest_cache/\nstack_outputs.json\nstack_outputs.env\n' >> .gitignore
```

Check your AWS credentials work from Python (this is the one place you touch real AWS in Parts 0 to G):

```
python3 -c "import boto3; print(boto3.client('sts').get_caller_identity()['Arn'])"
```

> 💡 boto3 finds credentials the same way the CLI does: environment variables, then `~/.aws/credentials`. Nothing you configured in earlier labs needs to change.

### 0.3. Create the folders

```
mkdir -p stack tests/unit tests/integration
touch stack/__init__.py tests/__init__.py tests/unit/__init__.py tests/integration/__init__.py
```

The empty `__init__.py` files make these folders importable packages.

Create `pytest.ini` in the `lab05` root:

```
[pytest]
testpaths = tests
markers =
    integration: runs against the stack you deployed in your own AWS account
    chaos: terminates a real instance and measures recovery (slow)
addopts = -m "not integration and not chaos" -ra
```

The `addopts` line is what makes a plain `pytest` run only the free unit tests. The integration and chaos tests must be asked for by name with `-m`.

> ⚠ **Run every command in this lab from inside `lab05/`**, with the virtual environment active (`source .venv/bin/activate`). The scripts read `.env` by relative path and the tests import the `stack` package from the current folder.

✅ **Checkpoint:** `pip list | grep -E "boto3|pytest|moto"` shows all three, and the `sts` command prints your IAM user's ARN.

---

## 6. Part A: From AWS CLI to boto3

### A1. The translation rules

Every CLI command is a boto3 method. The rules are mechanical:

| CLI | boto3 |
|---|---|
| `aws ec2 describe-vpcs` | `ec2 = boto3.client("ec2"); ec2.describe_vpcs()` |
| Service name (`ec2`, `elbv2`, `autoscaling`) | The argument to `boto3.client(...)` |
| Command with dashes (`describe-launch-templates`) | Method with underscores (`describe_launch_templates`) |
| `--launch-template-names foo` | `LaunchTemplateNames=["foo"]` (CamelCase, and lists are real lists) |
| `--filters "Name=isDefault,Values=true"` | `Filters=[{"Name": "isDefault", "Values": ["true"]}]` |
| `--query "Vpcs[0].VpcId" --output text` | `response["Vpcs"][0]["VpcId"]` |
| `--region us-east-2` | `boto3.client("ec2", region_name="us-east-2")`, once, when you create the client |
| `aws ec2 wait instance-running` | `client.get_waiter("instance_running").wait(...)` |
| Output over several pages | `client.get_paginator("list_objects_v2").paginate(...)` |
| `command 2>/dev/null \|\| true` | `try: ... except ClientError as err: if err.response["Error"]["Code"] != "Expected": raise` |

> 💡 Two tools make the translation easy. `aws ec2 describe-vpcs help` lists the options and their names. The boto3 documentation lists the same operations with the exact parameter names and response shapes. If you can read one, you can read the other.

### A2. One function, side by side

This is the bucket step from Lab03, in bash:

```bash
if aws s3api head-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" 2>/dev/null; then
  echo "Bucket already exists, skipping."
else
  aws s3api create-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" \
    --create-bucket-configuration LocationConstraint="$AWS_REGION"
fi
```

And in Python (you will write the real one in Part C):

```python
try:
    aws.s3.head_bucket(Bucket=name)
    log.info("Bucket already exists, skipping.")
except ClientError as err:
    if error_code(err) not in ("404", "NoSuchBucket", "NotFound"):
        raise                       # a 403 means someone else owns this name. Stop.
    aws.s3.create_bucket(Bucket=name, CreateBucketConfiguration={"LocationConstraint": region})
```

Look at what the Python version does that the bash version cannot. The bash `2>/dev/null` treats a **404** ("free to create") and a **403** ("another account owns this name") the same way, so a name collision with another student would fall through to `create-bucket` and fail with a confusing message. The Python version checks the error code, so it creates the bucket only when that is actually the right thing to do.

### A3. Errors are the main difference

The CLI collapses every failure into an exit code. boto3 raises `botocore.exceptions.ClientError`, and the specific problem is in `err.response["Error"]["Code"]`. Throughout this lab you will use two small helpers (Part B) built on that:

- `ignore_errors("EntityAlreadyExists")` is the narrow, honest version of `|| true`.
- `retry_on({"DependencyViolation"}, func)` retries only for errors that are expected to clear up on their own.

✅ **Checkpoint:** You can explain why `ignore_errors("NoSuchEntity")` is safer than `2>/dev/null || true`.

---

## 7. Part B: Configuration and Helpers

### B1. `stack/util.py`

```
        """Small helpers for working with AWS errors."""
        import time
        from contextlib import contextmanager

        from botocore.exceptions import ClientError


        def error_code(err: ClientError) -> str:
            return err.response.get("Error", {}).get("Code", "")


        @contextmanager
        def ignore_errors(*codes: str):
            """Swallow only the AWS error codes you expect, and re-raise everything else.

            This is the narrow version of bash's `2>/dev/null || true`, which hides
            every possible failure, including typos and permission problems.
            """
            try:
                yield
            except ClientError as err:
                if error_code(err) not in codes:
                    raise


        def retry_on(codes, func, attempts=10, delay=15, sleep=time.sleep):
            """Call func(); if AWS raises one of `codes`, wait and try again."""
            for attempt in range(1, attempts + 1):
                try:
                    return func()
                except ClientError as err:
                    if error_code(err) not in codes or attempt == attempts:
                        raise
                    sleep(delay)
```

### B2. `stack/config.py`

The settings come from the same `.env` file the bash scripts use. Nothing changes in `.env`. The `Tier` object groups everything that belongs to one tier, so the code never has to say "uploader" and "viewer" twice.

`stack/config.py`

```
        """Settings for the stack, read from the same .env file the bash scripts use."""
        from __future__ import annotations

        from dataclasses import dataclass
        from pathlib import Path

        REQUIRED_KEYS = [
            "AWS_REGION", "AMI_ID", "KEY_NAME", "INSTANCE_TYPE", "BUCKET_NAME",
            "APP_SECURITY_GROUP_ID",
            "UPLOADER_ROLE_NAME", "UPLOADER_PROFILE_NAME", "UPLOADER_LT_NAME",
            "UPLOADER_TG_NAME", "UPLOADER_ASG_NAME", "UPLOADER_LISTENER_PORT",
            "VIEWER_ROLE_NAME", "VIEWER_PROFILE_NAME", "VIEWER_LT_NAME",
            "VIEWER_TG_NAME", "VIEWER_ASG_NAME", "VIEWER_LISTENER_PORT",
            "ALB_NAME", "ALB_SG_NAME", "AZ_A", "AZ_B", "RECOVERY_SLO_SECONDS",
        ]


        class ConfigError(Exception):
            """Raised when .env is missing keys, with a message that says which."""


        def load_dotenv(path: str | Path = ".env") -> dict[str, str]:
            """Read KEY=VALUE lines into a dict. Comments and blank lines are skipped."""
            values: dict[str, str] = {}
            for raw in Path(path).read_text().splitlines():
                line = raw.strip()
                if not line or line.startswith("#") or "=" not in line:
```

```
                continue
            key, _, value = line.partition("=")
            values[key.strip()] = value.strip().strip('"').strip("'")
        return values


    @dataclass(frozen=True)
    class Tier:
        """Everything that belongs to one application tier (uploader or viewer)."""
        name: str
        role_name: str
        profile_name: str
        lt_name: str
        tg_name: str
        asg_name: str
        listener_port: int
        app_dir: Path

        @property
        def policy_name(self) -> str:
            return f"{self.role_name}-policy"

        @property
        def service_name(self) -> str:
            return self.name


    @dataclass(frozen=True)
    class Settings:
        region: str
        ami_id: str
        key_name: str
        instance_type: str
        bucket_name: str
        app_sg_id: str
        alb_name: str
        alb_sg_name: str
        az_a: str
        az_b: str
        recovery_slo_seconds: int
        uploader: Tier
        viewer: Tier
        app_port: int = 3000

        @property
        def tiers(self) -> tuple[Tier, Tier]:
            return (self.uploader, self.viewer)

        def tier(self, name: str) -> Tier:
            return {"uploader": self.uploader, "viewer": self.viewer}[name]

        @classmethod
        def from_values(cls, v: dict[str, str], base_dir: str | Path = ".") -> "Settings":
            missing = [k for k in REQUIRED_KEYS if not v.get(k)]
            if missing:
                raise ConfigError("Missing keys in .env: " + ", ".join(missing))
            base = Path(base_dir)

            def tier(prefix: str) -> Tier:
                return Tier(
                    name=prefix.lower(),
                    role_name=v[f"{prefix}_ROLE_NAME"],
                    profile_name=v[f"{prefix}_PROFILE_NAME"],
                    lt_name=v[f"{prefix}_LT_NAME"],
                    tg_name=v[f"{prefix}_TG_NAME"],
                    asg_name=v[f"{prefix}_ASG_NAME"],
                    listener_port=int(v[f"{prefix}_LISTENER_PORT"]),
                    app_dir=base / f"{prefix.lower()}-app",
                )

            return cls(
                region=v["AWS_REGION"], ami_id=v["AMI_ID"], key_name=v["KEY_NAME"],
                instance_type=v["INSTANCE_TYPE"], bucket_name=v["BUCKET_NAME"],
                app_sg_id=v["APP_SECURITY_GROUP_ID"], alb_name=v["ALB_NAME"],
```

```
                alb_sg_name=v["ALB_SG_NAME"], az_a=v["AZ_A"], az_b=v["AZ_B"],
                recovery_slo_seconds=int(v["RECOVERY_SLO_SECONDS"]),
                uploader=tier("UPLOADER"), viewer=tier("VIEWER"),
            )

        @classmethod
        def from_file(cls, path: str | Path = ".env") -> "Settings":
            path = Path(path)
            return cls.from_values(load_dotenv(path), base_dir=path.resolve().parent)
```

💡 `@dataclass(frozen=True)` gives you a class with named fields, a constructor and `==` for free, and `frozen` makes the settings read-only. A function can no longer change a setting by accident, which was easy to do with exported shell variables.

### B3. `stack/outputs.py`

The create step generates identifiers (the ALB's ARN and DNS name, subnet IDs). This class saves them as `stack_outputs.json` for Python, and also as `stack_outputs.env` so the bash scripts from earlier labs still work.

`stack/outputs.py`

```
"""Identifiers the create step generates, saved so tests and other tools can find them."""
from __future__ import annotations

import json
from dataclasses import asdict, dataclass
from pathlib import Path


@dataclass
class StackOutputs:
    vpc_id: str
    subnet_a: str
    subnet_b: str
    alb_security_group_id: str
    alb_arn: str
    alb_dns_name: str
    uploader_tg_arn: str
    viewer_tg_arn: str

    def tg_arn(self, tier_name: str) -> str:
        return {"uploader": self.uploader_tg_arn, "viewer": self.viewer_tg_arn}[tier_name]

    def save(self, directory: str | Path = ".") -> None:
        """Write stack_outputs.json (for Python) and stack_outputs.env (for the bash labs)."""
        directory = Path(directory)
        data = asdict(self)
        (directory / "stack_outputs.json").write_text(json.dumps(data, indent=2) + "\n")
        env_lines = [f"{key.upper()}={value}" for key, value in data.items()]
        (directory / "stack_outputs.env").write_text("\n".join(env_lines) + "\n")

    @classmethod
    def load(cls, directory: str | Path = ".") -> "StackOutputs":
        return cls(**json.loads((Path(directory) / "stack_outputs.json").read_text()))

    @staticmethod
    def remove(directory: str | Path = ".") -> None:
        for name in ("stack_outputs.json", "stack_outputs.env"):
            (Path(directory) / name).unlink(missing_ok=True)
```

### B4. `stack/clients.py`

```
"""One place that builds the boto3 clients the stack needs."""
from __future__ import annotations

from dataclasses import dataclass

import boto3
```

```
@dataclass
class AwsClients:
    s3: object
    iam: object
    ec2: object
    elbv2: object
    autoscaling: object

    @classmethod
    def create(cls, region: str) -> "AwsClients":
        session = boto3.session.Session(region_name=region)
        return cls(
            s3=session.client("s3"),
            iam=session.client("iam"),
            ec2=session.client("ec2"),
            elbv2=session.client("elbv2"),
            autoscaling=session.client("autoscaling"),
        )
```

### B5. `stack/policies.py`

IAM policies are plain dictionaries here. That makes them testable: Part G asserts that the uploader policy allows exactly one action on exactly one object.

`stack/policies.py`

```
"""IAM policy documents as plain Python dictionaries.

Keeping them as pure functions means we can test them without touching AWS.
"""
from __future__ import annotations

OBJECT_KEY = "shared.txt"


def trust_policy() -> dict:
    """Who may assume the role: the EC2 service, nobody else."""
    return {
        "Version": "2012-10-17",
        "Statement": [{
            "Effect": "Allow",
            "Principal": {"Service": "ec2.amazonaws.com"},
            "Action": "sts:AssumeRole",
        }],
    }


def uploader_policy(bucket: str) -> dict:
    """Write one object and nothing else."""
    return {
        "Version": "2012-10-17",
        "Statement": [{
            "Effect": "Allow",
            "Action": ["s3:PutObject"],
            "Resource": f"arn:aws:s3:::{bucket}/{OBJECT_KEY}",
        }],
    }


def viewer_policy(bucket: str) -> dict:
    """Read one object, and list the bucket."""
    return {
        "Version": "2012-10-17",
        "Statement": [
            {"Effect": "Allow", "Action": ["s3:GetObject"],
             "Resource": f"arn:aws:s3:::{bucket}/{OBJECT_KEY}"},
            {"Effect": "Allow", "Action": ["s3:ListBucket"],
             "Resource": f"arn:aws:s3:::{bucket}"},
        ],
    }
```

```
def policy_for(tier_name: str, bucket: str) -> dict:
    return {"uploader": uploader_policy, "viewer": viewer_policy}[tier_name](bucket)
```

✅ **Checkpoint:** `python3 -c "from stack.config import Settings; print(Settings.from_file('.env').tiers)"` prints two `Tier(...)` objects with your real names. If a key is missing, the error lists exactly which ones.

---

## 8. Part C: Storage and IAM

### C1. `stack/storage.py`

Compare each function with the matching block in `create_app_stack.sh`. The structure is the same: bucket, then a role, a policy and an instance profile for each tier. The difference is that one function, `ensure_role_and_profile`, now serves both tiers.

`stack/storage.py`

```
"""S3 bucket and IAM role and instance profile: the foundation of the stack."""
from __future__ import annotations

import json
import logging

from botocore.exceptions import ClientError

from .clients import AwsClients
from .config import Settings, Tier
from .policies import policy_for, trust_policy
from .util import error_code, ignore_errors

log = logging.getLogger("stack")


def ensure_bucket(aws: AwsClients, settings: Settings) -> None:
    try:
        aws.s3.head_bucket(Bucket=settings.bucket_name)
        log.info("Bucket %s already exists, skipping.", settings.bucket_name)
    except ClientError as err:
        # 404 means "free to create". 403 means someone else owns that name, so we must stop.
        if error_code(err) not in ("404", "NoSuchBucket", "NotFound"):
            raise
        args = {"Bucket": settings.bucket_name}
        if settings.region != "us-east-1":
            args["CreateBucketConfiguration"] = {"LocationConstraint": settings.region}
        aws.s3.create_bucket(**args)
        log.info("Created bucket %s", settings.bucket_name)
    # Not in the bash version: nothing in this lab ever needs a public bucket.
    aws.s3.put_public_access_block(
        Bucket=settings.bucket_name,
        PublicAccessBlockConfiguration={
            "BlockPublicAcls": True, "IgnorePublicAcls": True,
            "BlockPublicPolicy": True, "RestrictPublicBuckets": True,
        },
    )


def ensure_role_and_profile(aws: AwsClients, settings: Settings, tier: Tier) -> None:
    """Create the role, its inline policy, and an instance profile that holds the role."""
    with ignore_errors("EntityAlreadyExists"):
        aws.iam.create_role(
            RoleName=tier.role_name,
            AssumeRolePolicyDocument=json.dumps(trust_policy()),
        )
    aws.iam.put_role_policy(
        RoleName=tier.role_name,
        PolicyName=tier.policy_name,
        PolicyDocument=json.dumps(policy_for(tier.name, settings.bucket_name)),
    )
    with ignore_errors("EntityAlreadyExists"):
        aws.iam.create_instance_profile(InstanceProfileName=tier.profile_name)
```

```
        profile = aws.iam.get_instance_profile(InstanceProfileName=tier.profile_name)
        roles = [r["RoleName"] for r in profile["InstanceProfile"]["Roles"]]
        if tier.role_name not in roles:
            aws.iam.add_role_to_instance_profile(
                InstanceProfileName=tier.profile_name, RoleName=tier.role_name)
        log.info("%s role and instance profile are ready.", tier.name)


    def delete_role_and_profile(aws: AwsClients, tier: Tier) -> None:
        with ignore_errors("NoSuchEntity"):
            aws.iam.remove_role_from_instance_profile(
                InstanceProfileName=tier.profile_name, RoleName=tier.role_name)
        with ignore_errors("NoSuchEntity"):
            aws.iam.delete_instance_profile(InstanceProfileName=tier.profile_name)
        with ignore_errors("NoSuchEntity"):
            aws.iam.delete_role_policy(RoleName=tier.role_name, PolicyName=tier.policy_name)
        # A managed policy attached later (for example by a monitoring lab) would block delete_role.
        with ignore_errors("NoSuchEntity"):
            attached = aws.iam.list_attached_role_policies(RoleName=tier.role_name)
            for policy in attached["AttachedPolicies"]:
                aws.iam.detach_role_policy(RoleName=tier.role_name, PolicyArn=policy["PolicyArn"])
        with ignore_errors("NoSuchEntity"):
            aws.iam.delete_role(RoleName=tier.role_name)
        log.info("Removed %s role and instance profile.", tier.name)


    def delete_bucket(aws: AwsClients, settings: Settings) -> None:
        with ignore_errors("NoSuchBucket"):
            paginator = aws.s3.get_paginator("list_objects_v2")
            for page in paginator.paginate(Bucket=settings.bucket_name):
                objects = [{"Key": o["Key"]} for o in page.get("Contents", [])]
                if objects:
                    aws.s3.delete_objects(Bucket=settings.bucket_name, Delete={"Objects": objects})
            aws.s3.delete_bucket(Bucket=settings.bucket_name)
            log.info("Deleted bucket %s", settings.bucket_name)
```

Three things changed on purpose:

1. **The bucket blocks all public access.** Nothing in this lab ever needs a public bucket, so the code says so. The bash version did not set this.
2. **Instance profile membership is checked, not guessed.** The bash version ran `add-role-to-instance-profile ... || true` and hoped. This one reads the profile and adds the role only if it is missing.
3. **Delete detaches managed policies first.** An IAM role cannot be deleted while a managed policy is attached. A later lab attaches one, and this keeps cleanup working.

✓ **Checkpoint:** You can name which `ignore_errors` codes appear in `storage.py` and what each one means.

---

## 9. Part D: Launch Templates

### D1. `stack/compute.py`

`build_user_data` produces the same boot script as the bash version, using an f-string where bash used a heredoc with `$VARIABLES`. Separating it into a function that takes settings and returns a string is what makes it testable.

`stack/compute.py`

```
    """User-data and launch templates."""
    from __future__ import annotations

    import base64
    import logging

    from botocore.exceptions import ClientError

    from .clients import AwsClients
    from .config import Settings, Tier
    from .util import error_code
```

```
log = logging.getLogger("stack")


def _b64_file(path) -> str:
    return base64.b64encode(path.read_bytes()).decode()


def build_user_data(settings: Settings, tier: Tier) -> str:
    """The boot script: write the app files, install Node, run the app as a service."""
    app_js = _b64_file(tier.app_dir / "app.js")
    package_json = _b64_file(tier.app_dir / "package.json")
    return f"""#!/bin/bash
dnf update -y
dnf install -y nodejs
mkdir -p /opt/app
echo "{app_js}" | base64 -d > /opt/app/app.js
echo "{package_json}" | base64 -d > /opt/app/package.json
cd /opt/app
npm install --omit=dev

cat > /etc/systemd/system/{tier.service_name}.service << 'SERVICE'
[Unit]
Description={tier.name.capitalize()} App
After=network.target

[Service]
Environment=BUCKET_NAME={settings.bucket_name}
Environment=AWS_REGION={settings.region}
Environment=PORT={settings.app_port}
WorkingDirectory=/opt/app
ExecStart=/usr/bin/node /opt/app/app.js
Restart=always
User=ec2-user

[Install]
WantedBy=multi-user.target
SERVICE

chown -R ec2-user:ec2-user /opt/app
systemctl daemon-reload
systemctl enable {tier.service_name}.service
systemctl start {tier.service_name}.service
"""


def launch_template_data(settings: Settings, tier: Tier) -> dict:
    user_data = build_user_data(settings, tier)
    return {
        "ImageId": settings.ami_id,
        "InstanceType": settings.instance_type,
        "KeyName": settings.key_name,
        "SecurityGroupIds": [settings.app_sg_id],
        "IamInstanceProfile": {"Name": tier.profile_name},
        # The API wants user-data already base64 encoded.
        "UserData": base64.b64encode(user_data.encode()).decode(),
        "TagSpecifications": [{
            "ResourceType": "instance",
            "Tags": [{"Key": "Name", "Value": f"{tier.name}-app"}],
        }],
    }


def launch_template_exists(aws: AwsClients, tier: Tier) -> bool:
    try:
        aws.ec2.describe_launch_templates(LaunchTemplateNames=[tier.lt_name])
        return True
    except ClientError as err:
        if error_code(err) != "InvalidLaunchTemplateName.NotFoundException":
            raise
        return False


def ensure_launch_template(aws: AwsClients, settings: Settings, tier: Tier) -> int:
    """Create the template, or publish a new version if it already exists."""
```

```
    data = launch_template_data(settings, tier)
    if not launch_template_exists(aws, tier):
        aws.ec2.create_launch_template(LaunchTemplateName=tier.lt_name, LaunchTemplateData=data)
        log.info("Created launch template %s", tier.lt_name)
        return 1
    response = aws.ec2.create_launch_template_version(
        LaunchTemplateName=tier.lt_name, SourceVersion="1", LaunchTemplateData=data)
    version = response["LaunchTemplateVersion"]["VersionNumber"]
    log.info("Template %s exists, published version %s", tier.lt_name, version)
    return version


def delete_launch_template(aws: AwsClients, tier: Tier) -> None:
    if launch_template_exists(aws, tier):
        aws.ec2.delete_launch_template(LaunchTemplateName=tier.lt_name)
        log.info("Deleted launch template %s", tier.lt_name)
```

> 💡 `launch_template_exists` asks first and then acts. Python also makes it easy to return a value, so `ensure_launch_template` hands back the version number it published, and the tests use that.

---

## 10. Part E: Network, Load Balancer and Auto Scaling

### E1. `stack/network.py`

`StackError` is raised when the lab cannot continue (for example, your default VPC has no subnet in `us-east-2b`). A clear exception with a clear message replaces the bash pattern of printing `None` and carrying on.

`stack/network.py`

```
"""Default VPC, subnets, and the two security groups."""
from __future__ import annotations

import logging
import time

from .clients import AwsClients
from .config import Settings
from .util import ignore_errors, retry_on

log = logging.getLogger("stack")


class StackError(Exception):
    """Something the lab cannot continue without (for example, no subnet in an AZ)."""


def default_vpc_id(aws: AwsClients) -> str:
    vpcs = aws.ec2.describe_vpcs(Filters=[{"Name": "isDefault", "Values": ["true"]}])["Vpcs"]
    if not vpcs:
        raise StackError("No default VPC found in this region.")
    return vpcs[0]["VpcId"]


def subnet_in_az(aws: AwsClients, vpc_id: str, az: str) -> str:
    subnets = aws.ec2.describe_subnets(Filters=[
        {"Name": "vpc-id", "Values": [vpc_id]},
        {"Name": "availability-zone", "Values": [az]},
    ])["Subnets"]
    if not subnets:
        raise StackError(f"The default VPC has no subnet in {az}.")
    return subnets[0]["SubnetId"]


def _cidr_rule(port: int, cidr: str = "0.0.0.0/0") -> list[dict]:
    return [{"IpProtocol": "tcp", "FromPort": port, "ToPort": port,
             "IpRanges": [{"CidrIp": cidr}]}]
```

```
        def _group_rule(port: int, group_id: str) -> list[dict]:
            return [{"IpProtocol": "tcp", "FromPort": port, "ToPort": port,
                     "UserIdGroupPairs": [{"GroupId": group_id}]}]


        def find_security_group(aws: AwsClients, name: str, vpc_id: str) -> str | None:
            groups = aws.ec2.describe_security_groups(Filters=[
                {"Name": "group-name", "Values": [name]},
                {"Name": "vpc-id", "Values": [vpc_id]},
            ])["SecurityGroups"]
            return groups[0]["GroupId"] if groups else None


        def ensure_alb_security_group(aws: AwsClients, settings: Settings, vpc_id: str) -> str:
            group_id = find_security_group(aws, settings.alb_sg_name, vpc_id)
            if group_id is None:
                group_id = aws.ec2.create_security_group(
                    GroupName=settings.alb_sg_name,
                    Description="Allow inbound to the ALB on the app listener ports",
                    VpcId=vpc_id,
                )["GroupId"]
                log.info("Created ALB security group %s", group_id)
            for tier in settings.tiers:
                with ignore_errors("InvalidPermission.Duplicate"):
                    aws.ec2.authorize_security_group_ingress(
                        GroupId=group_id, IpPermissions=_cidr_rule(tier.listener_port))
            return group_id


        def restrict_app_security_group(aws: AwsClients, settings: Settings, alb_sg_id: str) -> None:
            """Replace 'port 3000 open to the world' with 'port 3000 open to the ALB only'."""
            with ignore_errors("InvalidPermission.NotFound"):
                aws.ec2.revoke_security_group_ingress(
                    GroupId=settings.app_sg_id, IpPermissions=_cidr_rule(settings.app_port))
            with ignore_errors("InvalidPermission.Duplicate"):
                aws.ec2.authorize_security_group_ingress(
                    GroupId=settings.app_sg_id, IpPermissions=_group_rule(settings.app_port, alb_sg_id))


        def restore_app_security_group(aws: AwsClients, settings: Settings, alb_sg_id: str | None) -> None:
            if alb_sg_id:
                with ignore_errors("InvalidPermission.NotFound"):
                    aws.ec2.revoke_security_group_ingress(
                        GroupId=settings.app_sg_id, IpPermissions=_group_rule(settings.app_port, alb_sg_id))
            with ignore_errors("InvalidPermission.Duplicate"):
                aws.ec2.authorize_security_group_ingress(
                    GroupId=settings.app_sg_id, IpPermissions=_cidr_rule(settings.app_port))


        def delete_alb_security_group(aws: AwsClients, group_id: str, sleep=time.sleep) -> None:
            # The load balancer's network interfaces can linger for a minute after it is deleted.
            with ignore_errors("InvalidGroup.NotFound"):
                retry_on({"DependencyViolation"},
                         lambda: aws.ec2.delete_security_group(GroupId=group_id),
                         attempts=10, delay=15, sleep=sleep)
                log.info("Deleted ALB security group %s", group_id)
```

### E2. `stack/balancer.py`

`HEALTH_CHECK` is a dictionary that is unpacked into the call with `**`. Notice the waiters: `get_waiter("load_balancer_available")` is the SDK form of `aws elbv2 wait load-balancer-available`.

`stack/balancer.py`

```
        """Target groups, the load balancer, and its listeners."""
        from __future__ import annotations

        import logging

        from botocore.exceptions import ClientError
```

```
        from .clients import AwsClients
        from .config import Settings, Tier
        from .util import error_code, ignore_errors

        log = logging.getLogger("stack")

        # Tighter than the defaults (30 seconds, 5 checks) so recovery is not dominated by health checks.
        HEALTH_CHECK = {
            "HealthCheckProtocol": "HTTP",
            "HealthCheckPath": "/",
            "HealthCheckIntervalSeconds": 10,
            "HealthCheckTimeoutSeconds": 5,
            "HealthyThresholdCount": 2,
            "UnhealthyThresholdCount": 2,
        }


        def find_target_group(aws: AwsClients, name: str) -> str | None:
            try:
                return aws.elbv2.describe_target_groups(Names=[name])["TargetGroups"][0]["TargetGroupArn"]
            except ClientError as err:
                if error_code(err) != "TargetGroupNotFound":
                    raise
                return None


        def ensure_target_group(aws: AwsClients, settings: Settings, tier: Tier, vpc_id: str) -> str:
            arn = find_target_group(aws, tier.tg_name)
            if arn:
                log.info("Target group %s already exists, skipping.", tier.tg_name)
                return arn
            response = aws.elbv2.create_target_group(
                Name=tier.tg_name, Protocol="HTTP", Port=settings.app_port, VpcId=vpc_id,
                TargetType="instance", **HEALTH_CHECK)
            return response["TargetGroups"][0]["TargetGroupArn"]


        def find_load_balancer(aws: AwsClients, name: str) -> dict | None:
            try:
                return aws.elbv2.describe_load_balancers(Names=[name])["LoadBalancers"][0]
            except ClientError as err:
                if error_code(err) != "LoadBalancerNotFound":
                    raise
                return None


        def ensure_load_balancer(aws: AwsClients, settings: Settings, subnets: list[str],
                                 alb_sg_id: str) -> tuple[str, str]:
            """Return (arn, dns_name), creating the ALB and waiting for it to be active if needed."""
            lb = find_load_balancer(aws, settings.alb_name)
            if lb:
                log.info("Load balancer %s already exists, skipping.", settings.alb_name)
                return lb["LoadBalancerArn"], lb["DNSName"]
            lb = aws.elbv2.create_load_balancer(
                Name=settings.alb_name, Subnets=subnets, SecurityGroups=[alb_sg_id],
                Scheme="internet-facing", Type="application")["LoadBalancers"][0]
            log.info("Waiting for the ALB to become active...")
            aws.elbv2.get_waiter("load_balancer_available").wait(LoadBalancerArns=[lb["LoadBalancerArn"]])
            return lb["LoadBalancerArn"], lb["DNSName"]


        def ensure_listener(aws: AwsClients, alb_arn: str, port: int, tg_arn: str) -> None:
            existing = aws.elbv2.describe_listeners(LoadBalancerArn=alb_arn)["Listeners"]
            if any(listener["Port"] == port for listener in existing):
                log.info("Listener on port %s already exists, skipping.", port)
                return
            aws.elbv2.create_listener(
                LoadBalancerArn=alb_arn, Protocol="HTTP", Port=port,
                DefaultActions=[{"Type": "forward", "TargetGroupArn": tg_arn}])


        def delete_load_balancer(aws: AwsClients, name: str) -> None:
            lb = find_load_balancer(aws, name)
            if not lb:
```

```
            return
        aws.elbv2.delete_load_balancer(LoadBalancerArn=lb["LoadBalancerArn"])
        aws.elbv2.get_waiter("load_balancers_deleted").wait(LoadBalancerArns=[lb["LoadBalancerArn"]])
        log.info("Deleted load balancer %s", name)


    def delete_target_group(aws: AwsClients, name: str) -> None:
        arn = find_target_group(aws, name)
        if arn:
            with ignore_errors("TargetGroupNotFound"):
                aws.elbv2.delete_target_group(TargetGroupArn=arn)
                log.info("Deleted target group %s", name)
```

### E3. `stack/scaling.py`

`delete_asg` replaces the 20-iteration bash loop. The `sleep` argument is injectable, which lets a test pass in a function that does nothing instead of waiting.

`stack/scaling.py`

```
    """Auto Scaling Groups: the part that launches (and relaunches) the instances."""
    from __future__ import annotations

    import logging
    import time

    from .clients import AwsClients
    from .config import Settings, Tier

    log = logging.getLogger("stack")


    def find_asg(aws: AwsClients, name: str) -> dict | None:
        groups = aws.autoscaling.describe_auto_scaling_groups(
            AutoScalingGroupNames=[name])["AutoScalingGroups"]
        return groups[0] if groups else None


    def ensure_asg(aws: AwsClients, tier: Tier, tg_arn: str, subnets: list[str]) -> None:
        if find_asg(aws, tier.asg_name):
            log.info("Auto Scaling Group %s already exists, skipping.", tier.asg_name)
            return
        aws.autoscaling.create_auto_scaling_group(
            AutoScalingGroupName=tier.asg_name,
            LaunchTemplate={"LaunchTemplateName": tier.lt_name, "Version": "$Latest"},
            MinSize=1, MaxSize=1, DesiredCapacity=1,
            VPCZoneIdentifier=",".join(subnets),
            TargetGroupARNs=[tg_arn],
            HealthCheckType="ELB",
            HealthCheckGracePeriod=120,
            Tags=[{
                "ResourceId": tier.asg_name, "ResourceType": "auto-scaling-group",
                "Key": "Name", "Value": f"{tier.name}-app", "PropagateAtLaunch": True,
            }],
        )
        log.info("Created Auto Scaling Group %s", tier.asg_name)


    def delete_asg(aws: AwsClients, name: str, timeout: int = 300, poll: int = 15,
                   sleep=time.sleep) -> None:
        """Force-delete the group (which terminates its instances) and wait until it is gone."""
        if not find_asg(aws, name):
            return
        aws.autoscaling.delete_auto_scaling_group(AutoScalingGroupName=name, ForceDelete=True)
        waited = 0
        while find_asg(aws, name):
            if waited >= timeout:
                log.warning("%s still deleting after %ss. Check the console.", name, timeout)
                return
            sleep(poll)
            waited += poll
        log.info("Deleted Auto Scaling Group %s", name)
```

✅ **Checkpoint:** You can point to the line in `ensure_asg` that makes the ASG trust the load balancer's health check, and say why that matters for recovery.

---

## 11. Part F: Putting It Together

### F1. `stack/runner.py`

Read `create_stack` top to bottom. It is the whole of `create_app_stack.sh` in about 40 lines, because every step is a function that already exists.

`delete_stack` finds resources **by name**, not from saved state. That means it still works after a half-finished create, or if you deleted `stack_outputs.json` by accident.

`stack/runner.py`

```
        """The two entry points: build the whole stack, and tear it down."""
        from __future__ import annotations

        import logging
        import time

        from . import balancer, compute, network, scaling, storage
        from .clients import AwsClients
        from .config import Settings
        from .outputs import StackOutputs

        log = logging.getLogger("stack")


        def create_stack(settings: Settings, aws: AwsClients, iam_wait: int = 15,
                         sleep=time.sleep) -> StackOutputs:
            log.info("== S3 bucket ==")
            storage.ensure_bucket(aws, settings)

            log.info("== IAM roles and instance profiles ==")
            for tier in settings.tiers:
                storage.ensure_role_and_profile(aws, settings, tier)
            log.info("Waiting %ss for IAM propagation...", iam_wait)
            sleep(iam_wait)

            log.info("== Launch templates ==")
            for tier in settings.tiers:
                compute.ensure_launch_template(aws, settings, tier)

            log.info("== Network ==")
            vpc_id = network.default_vpc_id(aws)
            subnet_a = network.subnet_in_az(aws, vpc_id, settings.az_a)
            subnet_b = network.subnet_in_az(aws, vpc_id, settings.az_b)
            alb_sg_id = network.ensure_alb_security_group(aws, settings, vpc_id)
            network.restrict_app_security_group(aws, settings, alb_sg_id)

            log.info("== Target groups, load balancer, listeners ==")
            tg_arns = {tier.name: balancer.ensure_target_group(aws, settings, tier, vpc_id)
                       for tier in settings.tiers}
            alb_arn, alb_dns = balancer.ensure_load_balancer(aws, settings, [subnet_a, subnet_b], alb_sg_id)
            for tier in settings.tiers:
                balancer.ensure_listener(aws, alb_arn, tier.listener_port, tg_arns[tier.name])

            log.info("== Auto Scaling Groups ==")
            for tier in settings.tiers:
                scaling.ensure_asg(aws, tier, tg_arns[tier.name], [subnet_a, subnet_b])

            return StackOutputs(
                vpc_id=vpc_id, subnet_a=subnet_a, subnet_b=subnet_b,
                alb_security_group_id=alb_sg_id, alb_arn=alb_arn, alb_dns_name=alb_dns,
                uploader_tg_arn=tg_arns["uploader"], viewer_tg_arn=tg_arns["viewer"],
            )
```

```
    def delete_stack(settings: Settings, aws: AwsClients, keep_foundation: bool = False,
                     sleep=time.sleep) -> None:
        """Delete by name, not by saved state, so it still works after a half-finished create."""
        log.info("== Auto Scaling Groups (this also terminates their instances) ==")
        for tier in settings.tiers:
            scaling.delete_asg(aws, tier.asg_name, sleep=sleep)

        log.info("== Load balancer and target groups ==")
        balancer.delete_load_balancer(aws, settings.alb_name)
        for tier in settings.tiers:
            balancer.delete_target_group(aws, tier.tg_name)

        log.info("== Security groups ==")
        vpc_id = network.default_vpc_id(aws)
        alb_sg_id = network.find_security_group(aws, settings.alb_sg_name, vpc_id)
        network.restore_app_security_group(aws, settings, alb_sg_id)
        if alb_sg_id:
            network.delete_alb_security_group(aws, alb_sg_id, sleep=sleep)

        if keep_foundation:
            log.info("Keeping the bucket, IAM roles and launch templates (--keep-foundation).")
        else:
            log.info("== Launch templates, IAM, bucket ==")
            for tier in settings.tiers:
                compute.delete_launch_template(aws, tier)
                storage.delete_role_and_profile(aws, tier)
            storage.delete_bucket(aws, settings)

        StackOutputs.remove()
        log.info("Cleanup complete.")
```

## F2. `create_stack.py` and `delete_stack.py`

These are thin command-line wrappers. All of the logic lives in the package, so the tests can call it directly.

`create_stack.py`

```
#!/usr/bin/env python3
"""Build the whole stack: the Python replacement for create_app_stack.sh."""
import argparse
import logging

from stack.clients import AwsClients
from stack.config import Settings
from stack.runner import create_stack


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--env", default=".env", help="path to the settings file")
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(message)s")

    settings = Settings.from_file(args.env)
    outputs = create_stack(settings, AwsClients.create(settings.region))
    outputs.save()

    print()
    print(f"Uploader: http://{outputs.alb_dns_name}:{settings.uploader.listener_port}/")
    print(f"Viewer:   http://{outputs.alb_dns_name}:{settings.viewer.listener_port}/")
    print("Give the target groups 1 to 2 minutes to report healthy, then run: pytest -m integration")


if __name__ == "__main__":
    main()
```

`delete_stack.py`

```
#!/usr/bin/env python3
"""Tear the stack down: the Python replacement for delete_app_stack.sh."""
```

```
import argparse
import logging

from stack.clients import AwsClients
from stack.config import Settings
from stack.runner import delete_stack


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--env", default=".env", help="path to the settings file")
    parser.add_argument("--keep-foundation", action="store_true",
                        help="keep the bucket, IAM roles and launch templates")
    args = parser.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(message)s")

    settings = Settings.from_file(args.env)
    delete_stack(settings, AwsClients.create(settings.region), keep_foundation=args.keep_foundation)


if __name__ == "__main__":
    main()
```

```
chmod +x create_stack.py delete_stack.py
```

✅ **Checkpoint:** `python3 -m py_compile stack/*.py create_stack.py delete_stack.py` prints nothing. Do not run `create_stack.py` yet. The unit tests come first, and they are free.

---

## 12. Part G: Unit Tests (No AWS Account Needed)

### G1. Shared fixtures

`fake_settings` builds a complete `Settings` object pointing at two tiny fake app folders, so unit tests never need your real `.env`. The three fixtures at the bottom are for the integration tests later.

`tests/conftest.py`

```
"""Shared fixtures. Unit tests use fake settings, integration tests use your real .env."""
from pathlib import Path

import pytest

from stack.clients import AwsClients
from stack.config import Settings
from stack.outputs import StackOutputs

ROOT = Path(__file__).resolve().parent.parent

FAKE_VALUES = {
    "AWS_REGION": "us-east-2", "AMI_ID": "ami-12345678", "KEY_NAME": "test-key",
    "INSTANCE_TYPE": "t3.micro", "BUCKET_NAME": "unit-test-bucket-12345",
    "APP_SECURITY_GROUP_ID": "sg-placeholder",
    "UPLOADER_ROLE_NAME": "t-uploader-role", "UPLOADER_PROFILE_NAME": "t-uploader-profile",
    "UPLOADER_LT_NAME": "t-uploader-lt", "UPLOADER_TG_NAME": "t-uploader-tg",
    "UPLOADER_ASG_NAME": "t-uploader-asg", "UPLOADER_LISTENER_PORT": "8080",
    "VIEWER_ROLE_NAME": "t-viewer-role", "VIEWER_PROFILE_NAME": "t-viewer-profile",
    "VIEWER_LT_NAME": "t-viewer-lt", "VIEWER_TG_NAME": "t-viewer-tg",
    "VIEWER_ASG_NAME": "t-viewer-asg", "VIEWER_LISTENER_PORT": "8081",
    "ALB_NAME": "t-alb", "ALB_SG_NAME": "t-alb-sg",
    "AZ_A": "us-east-2a", "AZ_B": "us-east-2b", "RECOVERY_SLO_SECONDS": "180",
}


@pytest.fixture
def fake_settings(tmp_path):
    """Settings pointing at two tiny fake app folders, so no real files are needed."""
    for name in ("uploader-app", "viewer-app"):
        folder = tmp_path / name
```

```
            folder.mkdir()
            (folder / "app.js").write_text(f"console.log('{name}');\n")
            (folder / "package.json").write_text('{"name": "%s"}\n' % name)
        return Settings.from_values(FAKE_VALUES, base_dir=tmp_path)


    # ---- fixtures for tests that talk to your deployed stack ----

    @pytest.fixture(scope="session")
    def settings():
        return Settings.from_file(ROOT / ".env")


    @pytest.fixture(scope="session")
    def aws(settings):
        return AwsClients.create(settings.region)


    @pytest.fixture(scope="session")
    def outputs():
        try:
            return StackOutputs.load(ROOT)
        except FileNotFoundError:
            pytest.skip("stack_outputs.json not found. Run python create_stack.py first.")
```

## G2. Test the configuration

`tests/unit/test_config.py`

```
    import pytest

    from stack.config import ConfigError, Settings, load_dotenv
    from tests.conftest import FAKE_VALUES


    def test_load_dotenv_skips_comments_and_blank_lines(tmp_path):
        env = tmp_path / ".env"
        env.write_text("# a comment\n\nAWS_REGION=us-east-2\nKEY_NAME='quoted'\n")
        assert load_dotenv(env) == {"AWS_REGION": "us-east-2", "KEY_NAME": "quoted"}


    def test_missing_keys_are_named_in_the_error():
        values = dict(FAKE_VALUES)
        del values["ALB_NAME"]
        del values["AZ_B"]
        with pytest.raises(ConfigError) as err:
            Settings.from_values(values)
        assert "ALB_NAME" in str(err.value) and "AZ_B" in str(err.value)


    def test_tiers_are_built_from_prefixed_keys(fake_settings):
        up, vw = fake_settings.tiers
        assert (up.name, up.listener_port, up.asg_name) == ("uploader", 8080, "t-uploader-asg")
        assert (vw.name, vw.listener_port, vw.lt_name) == ("viewer", 8081, "t-viewer-lt")
        assert up.policy_name == "t-uploader-role-policy"
        assert up.app_dir.name == "uploader-app"


    def test_ports_are_integers(fake_settings):
        assert isinstance(fake_settings.uploader.listener_port, int)
        assert isinstance(fake_settings.recovery_slo_seconds, int)
```

## G3. Test the IAM policies (least privilege, as code)

These tests encode your security design. If someone later "fixes" a permissions error by adding `s3:*`, a test fails and tells them why.

`tests/unit/test_policies.py`

```
    """Least privilege, checked as code. These tests need no AWS account."""
    from stack.policies import policy_for, trust_policy
```

```
BUCKET = "my-bucket"


def _allowed_actions(policy):
    actions = []
    for statement in policy["Statement"]:
        assert statement["Effect"] == "Allow"
        actions += statement["Action"]
    return actions


def test_trust_policy_only_trusts_ec2():
    statement = trust_policy()["Statement"][0]
    assert statement["Principal"] == {"Service": "ec2.amazonaws.com"}
    assert statement["Action"] == "sts:AssumeRole"


def test_uploader_can_only_put_one_object():
    policy = policy_for("uploader", BUCKET)
    assert _allowed_actions(policy) == ["s3:PutObject"]
    assert policy["Statement"][0]["Resource"] == f"arn:aws:s3:::{BUCKET}/shared.txt"


def test_viewer_can_read_but_never_write():
    actions = _allowed_actions(policy_for("viewer", BUCKET))
    assert "s3:GetObject" in actions
    assert not any(a in actions for a in ("s3:PutObject", "s3:DeleteObject", "s3:*", "*"))


def test_no_policy_uses_a_wildcard_resource():
    for tier in ("uploader", "viewer"):
        for statement in policy_for(tier, BUCKET)["Statement"]:
            assert statement["Resource"] != "*"
```

## G4. Test the user-data

The round-trip test proves the files embedded in the boot script decode back to exactly your app code.

`tests/unit/test_user_data.py`

```
import base64
import re

from stack.compute import build_user_data, launch_template_data


def test_app_files_are_embedded_and_round_trip(fake_settings):
    tier = fake_settings.uploader
    script = build_user_data(fake_settings, tier)
    encoded = re.findall(r'echo "([A-Za-z0-9+/=]+)" \| base64 -d', script)
    decoded = [base64.b64decode(item).decode() for item in encoded]
    assert decoded[0] == (tier.app_dir / "app.js").read_text()
    assert decoded[1] == (tier.app_dir / "package.json").read_text()


def test_service_gets_the_right_environment(fake_settings):
    script = build_user_data(fake_settings, fake_settings.viewer)
    assert "Environment=BUCKET_NAME=unit-test-bucket-12345" in script
    assert "Environment=AWS_REGION=us-east-2" in script
    assert "systemctl start viewer.service" in script
    assert "Restart=always" in script


def test_script_starts_with_a_shebang(fake_settings):
    assert build_user_data(fake_settings, fake_settings.uploader).startswith("#!/bin/bash\n")


def test_launch_template_user_data_is_base64_of_the_script(fake_settings):
    data = launch_template_data(fake_settings, fake_settings.viewer)
    decoded = base64.b64decode(data["UserData"]).decode()
    assert decoded == build_user_data(fake_settings, fake_settings.viewer)
```

```
        assert data["IamInstanceProfile"] == {"Name": "t-viewer-profile"}
        assert data["SecurityGroupIds"] == ["sg-placeholder"]
```

## G5. Test the whole stack against a fake AWS

`mock_aws()` replaces AWS with an in-memory fake for everything inside the `with` block. The code under test is your real `create_stack` and `delete_stack`, unchanged. The `env` fixture first builds the things Lab03 would already have created (the app security group, the key pair, an AMI id).

`tests/unit/test_stack_with_moto.py`

```
    """Run the real create and delete code against moto, an in-memory fake of AWS.

    Nothing here touches your AWS account, costs money, or needs credentials.
    """
    import json

    import pytest
    from moto import mock_aws

    from stack.clients import AwsClients
    from stack.config import Settings
    from stack.runner import create_stack, delete_stack
    from tests.conftest import FAKE_VALUES


    @pytest.fixture
    def env(tmp_path, monkeypatch, fake_settings):
        """A fake AWS account with the things lab03 would already have created."""
        monkeypatch.chdir(tmp_path)  # stack_outputs files are written to the current folder
        with mock_aws():
            aws = AwsClients.create("us-east-2")
            vpc = aws.ec2.describe_vpcs(Filters=[{"Name": "isDefault", "Values": ["true"]}])["Vpcs"][0]
            sg = aws.ec2.create_security_group(
                GroupName="app-sg", Description="app", VpcId=vpc["VpcId"])["GroupId"]
            aws.ec2.authorize_security_group_ingress(GroupId=sg, IpPermissions=[{
                "IpProtocol": "tcp", "FromPort": 3000, "ToPort": 3000,
                "IpRanges": [{"CidrIp": "0.0.0.0/0"}]}])
            aws.ec2.create_key_pair(KeyName="test-key")
            ami = aws.ec2.describe_images()["Images"][0]["ImageId"]
            values = dict(FAKE_VALUES, APP_SECURITY_GROUP_ID=sg, AMI_ID=ami)
            settings = Settings.from_values(values, base_dir=fake_settings.uploader.app_dir.parent)
            yield settings, aws


    def _app_sg_rules(aws, settings):
        group = aws.ec2.describe_security_groups(GroupIds=[settings.app_sg_id])["SecurityGroups"][0]
        return [p for p in group["IpPermissions"] if p["FromPort"] == 3000]


    def test_create_builds_every_resource(env):
        settings, aws = env
        outputs = create_stack(settings, aws, iam_wait=0)

        aws.s3.head_bucket(Bucket=settings.bucket_name)
        block = aws.s3.get_public_access_block(Bucket=settings.bucket_name)["PublicAccessBlockConfiguration"]
        assert all(block.values())

        for tier in settings.tiers:
            role_policy = aws.iam.get_role_policy(RoleName=tier.role_name, PolicyName=tier.policy_name)
            assert role_policy["PolicyDocument"]["Statement"]
            profile = aws.iam.get_instance_profile(InstanceProfileName=tier.profile_name)
            assert [r["RoleName"] for r in profile["InstanceProfile"]["Roles"]] == [tier.role_name]
            assert aws.ec2.describe_launch_templates(LaunchTemplateNames=[tier.lt_name])["LaunchTemplates"]

            group = aws.autoscaling.describe_auto_scaling_groups(
                AutoScalingGroupNames=[tier.asg_name])["AutoScalingGroups"][0]
            assert (group["MinSize"], group["MaxSize"], group["DesiredCapacity"]) == (1, 1, 1)
            assert group["HealthCheckType"] == "ELB"

            tg = aws.elbv2.describe_target_groups(Names=[tier.tg_name])["TargetGroups"][0]
            assert tg["HealthCheckIntervalSeconds"] == 10
```

```
        assert tg["UnhealthyThresholdCount"] == 2

    listeners = aws.elbv2.describe_listeners(LoadBalancerArn=outputs.alb_arn)["Listeners"]
    assert sorted(item["Port"] for item in listeners) == [8080, 8081]
    assert outputs.alb_dns_name


def test_app_security_group_only_accepts_the_alb(env):
    settings, aws = env
    outputs = create_stack(settings, aws, iam_wait=0)
    rules = _app_sg_rules(aws, settings)
    assert len(rules) == 1
    assert rules[0]["IpRanges"] == []
    assert rules[0]["UserIdGroupPairs"][0]["GroupId"] == outputs.alb_security_group_id


def test_create_twice_does_not_duplicate_anything(env):
    settings, aws = env
    first = create_stack(settings, aws, iam_wait=0)
    second = create_stack(settings, aws, iam_wait=0)
    assert first == second
    assert len(aws.elbv2.describe_load_balancers()["LoadBalancers"]) == 1
    assert len(aws.elbv2.describe_target_groups()["TargetGroups"]) == 2
    assert len(aws.autoscaling.describe_auto_scaling_groups()["AutoScalingGroups"]) == 2
    assert len(_app_sg_rules(aws, settings)) == 1
    # a second run publishes a new launch template version instead of failing
    versions = aws.ec2.describe_launch_template_versions(
        LaunchTemplateName=settings.uploader.lt_name)["LaunchTemplateVersions"]
    assert len(versions) == 2


def test_delete_removes_everything_and_restores_the_app_sg(env):
    settings, aws = env
    create_stack(settings, aws, iam_wait=0).save()
    delete_stack(settings, aws, sleep=lambda _: None)

    assert aws.elbv2.describe_load_balancers()["LoadBalancers"] == []
    assert aws.elbv2.describe_target_groups()["TargetGroups"] == []
    assert aws.autoscaling.describe_auto_scaling_groups()["AutoScalingGroups"] == []
    assert aws.ec2.describe_launch_templates()["LaunchTemplates"] == []
    assert aws.iam.list_roles()["Roles"] == []
    assert aws.iam.list_instance_profiles()["InstanceProfiles"] == []
    assert settings.bucket_name not in [b["Name"] for b in aws.s3.list_buckets()["Buckets"]]
    rules = _app_sg_rules(aws, settings)
    assert rules[0]["IpRanges"] == [{"CidrIp": "0.0.0.0/0"}]
    assert rules[0]["UserIdGroupPairs"] == []


def test_delete_is_safe_to_run_twice(env):
    settings, aws = env
    create_stack(settings, aws, iam_wait=0)
    delete_stack(settings, aws, sleep=lambda _: None)
    delete_stack(settings, aws, sleep=lambda _: None)  # nothing left, must not raise


def test_keep_foundation_leaves_bucket_roles_and_templates(env):
    settings, aws = env
    create_stack(settings, aws, iam_wait=0)
    delete_stack(settings, aws, keep_foundation=True, sleep=lambda _: None)
    assert aws.autoscaling.describe_auto_scaling_groups()["AutoScalingGroups"] == []
    assert len(aws.iam.list_roles()["Roles"]) == 2
    assert len(aws.ec2.describe_launch_templates()["LaunchTemplates"]) == 2
    aws.s3.head_bucket(Bucket=settings.bucket_name)


def test_delete_detaches_extra_managed_policies(env):
    settings, aws = env
    create_stack(settings, aws, iam_wait=0)
    extra = aws.iam.create_policy(PolicyName="extra", PolicyDocument=json.dumps({
        "Version": "2012-10-17",
        "Statement": [{"Effect": "Allow", "Action": "logs:PutLogEvents", "Resource": "*"}]}))
    aws.iam.attach_role_policy(RoleName=settings.uploader.role_name,
                               PolicyArn=extra["Policy"]["Arn"])
    delete_stack(settings, aws, sleep=lambda _: None)
```

```
        assert aws.iam.list_roles()["Roles"] == []
```

Run everything:

```
    pytest
```

Expected: **19 passed** in roughly 15 seconds, with no AWS calls made. Try these to see the tests do real work:

1. In `stack/policies.py`, change the uploader's `"s3:PutObject"` to `"s3:*"` and run `pytest`. Watch `test_uploader_can_only_put_one_object` fail. Revert it.
2. In `stack/scaling.py`, change `HealthCheckType="ELB"` to `"EC2"`. Which test fails, and what does its message say? Revert it.
3. In `stack/network.py`, comment out the call to `restrict_app_security_group` in `runner.py`. Which test catches it?

### G6. What moto can and cannot tell you

| Moto proves | Moto cannot prove |
|---|---|
| Your code calls the AWS APIs with valid parameters in a sensible order | That real AWS accepts those parameters (IAM propagation delays, service quotas, regional differences) |
| Create is idempotent, delete is safe to repeat, rollback paths work | That the instances boot, `npm install` succeeds, or the app answers HTTP |
| Resources end up with the configuration you intended | That your IAM policies actually allow and deny what you think they do |

The right-hand column is why the next part exists.

✅ **Checkpoint:** `pytest` reports 19 passed, and you completed at least one of the "break it on purpose" experiments.

---

## 13. Part H: Integration and Chaos Tests (Against Your Real Stack)

### H1. Helpers

`tests/helpers.py`

```
    """Small helpers shared by the integration tests."""
    import time

    import requests


    def probe(url: str, timeout: float = 3.0) -> bool:
        """True if the URL answers 200 within the timeout."""
        try:
            return requests.get(url, timeout=timeout).status_code == 200
        except requests.RequestException:
            return False


    def wait_until(check, timeout: int = 300, interval: int = 10):
        """Call check() until it returns something truthy, or fail after `timeout` seconds."""
        deadline = time.time() + timeout
        while time.time() < deadline:
            result = check()
            if result:
                return result
            time.sleep(interval)
        return None


    def tier_url(outputs, settings, tier_name: str) -> str:
        port = settings.tier(tier_name).listener_port
        return f"http://{outputs.alb_dns_name}:{port}/"
```

### H2. Check the infrastructure

Each test is a statement about the design. Notice `test_uploader_role_can_only_write_the_shared_object`: it uses **`iam.simulate_principal_policy`** to ask AWS itself what the role is allowed to do, instead of trusting the policy text you wrote. That is a security test, not a unit test.

`tests/integration/test_infrastructure.py`

```
        """Does the stack you deployed look the way the design says it should?

        Run after create_stack.py:   pytest -m integration
        """
        import pytest

        from tests.helpers import wait_until

        pytestmark = pytest.mark.integration


        def test_bucket_blocks_all_public_access(aws, settings):
            block = aws.s3.get_public_access_block(
                Bucket=settings.bucket_name)["PublicAccessBlockConfiguration"]
            assert all(block.values())


        def _decision(aws, role_name, action, resource):
            role_arn = aws.iam.get_role(RoleName=role_name)["Role"]["Arn"]
            result = aws.iam.simulate_principal_policy(
                PolicySourceArn=role_arn, ActionNames=[action], ResourceArns=[resource])
            return result["EvaluationResults"][0]["EvalDecision"]


        def test_uploader_role_can_only_write_the_shared_object(aws, settings):
            role = settings.uploader.role_name
            shared = f"arn:aws:s3:::{settings.bucket_name}/shared.txt"
            other = f"arn:aws:s3:::{settings.bucket_name}/other.txt"
            assert _decision(aws, role, "s3:PutObject", shared) == "allowed"
            assert _decision(aws, role, "s3:PutObject", other) != "allowed"
            assert _decision(aws, role, "s3:GetObject", shared) != "allowed"


        def test_viewer_role_can_read_but_not_write(aws, settings):
            role = settings.viewer.role_name
            shared = f"arn:aws:s3:::{settings.bucket_name}/shared.txt"
            assert _decision(aws, role, "s3:GetObject", shared) == "allowed"
            assert _decision(aws, role, "s3:PutObject", shared) != "allowed"
            assert _decision(aws, role, "s3:DeleteObject", shared) != "allowed"


        def test_load_balancer_is_public_and_spans_two_azs(aws, settings, outputs):
            lb = aws.elbv2.describe_load_balancers(LoadBalancerArns=[outputs.alb_arn])["LoadBalancers"][0]
            assert lb["State"]["Code"] == "active"
            assert lb["Scheme"] == "internet-facing"
            zones = {az["ZoneName"] for az in lb["AvailabilityZones"]}
            assert zones == {settings.az_a, settings.az_b}


        def test_listeners_forward_to_the_right_target_groups(aws, settings, outputs):
            listeners = aws.elbv2.describe_listeners(LoadBalancerArn=outputs.alb_arn)["Listeners"]
            forwards = {item["Port"]: item["DefaultActions"][0]["TargetGroupArn"] for item in listeners}
            for tier in settings.tiers:
                assert forwards[tier.listener_port] == outputs.tg_arn(tier.name)


        def test_target_groups_use_fast_health_checks(aws, settings, outputs):
            for tier in settings.tiers:
                tg = aws.elbv2.describe_target_groups(
                    TargetGroupArns=[outputs.tg_arn(tier.name)])["TargetGroups"][0]
                assert tg["HealthCheckPath"] == "/"
                assert tg["HealthCheckIntervalSeconds"] == 10
                assert tg["HealthyThresholdCount"] == 2


        def test_auto_scaling_groups_are_configured_for_recovery(aws, settings, outputs):
            for tier in settings.tiers:
```

```
            group = aws.autoscaling.describe_auto_scaling_groups(
                AutoScalingGroupNames=[tier.asg_name])["AutoScalingGroups"][0]
            assert group["HealthCheckType"] == "ELB"
            assert group["DesiredCapacity"] == 1
            assert set(group["VPCZoneIdentifier"].split(",")) == {outputs.subnet_a, outputs.subnet_b}
            assert outputs.tg_arn(tier.name) in group["TargetGroupARNs"]


    def test_app_port_is_reachable_only_from_the_load_balancer(aws, settings, outputs):
        group = aws.ec2.describe_security_groups(GroupIds=[settings.app_sg_id])["SecurityGroups"][0]
        rules = [p for p in group["IpPermissions"] if p.get("FromPort") == settings.app_port]
        assert rules, "port 3000 rule missing"
        for rule in rules:
            assert rule["IpRanges"] == [], "port 3000 must not be open to a CIDR range"
            assert [p["GroupId"] for p in rule["UserIdGroupPairs"]] == [outputs.alb_security_group_id]


    def test_every_target_becomes_healthy(aws, settings, outputs):
        def all_healthy():
            for tier in settings.tiers:
                targets = aws.elbv2.describe_target_health(
                    TargetGroupArn=outputs.tg_arn(tier.name))["TargetHealthDescriptions"]
                if not targets or any(t["TargetHealth"]["State"] != "healthy" for t in targets):
                    return False
            return True

        assert wait_until(all_healthy, timeout=420, interval=15), "targets never became healthy"
```

## H3. Check the application through the load balancer

`tests/integration/test_application.py`

```
    """Do the two apps behave correctly when you use them through the load balancer?

    Run after the targets are healthy:   pytest -m integration
    """
    import uuid

    import pytest
    import requests

    from tests.helpers import tier_url, wait_until

    pytestmark = pytest.mark.integration


    @pytest.fixture(scope="module")
    def uploader_url(settings, outputs):
        return tier_url(outputs, settings, "uploader")


    @pytest.fixture(scope="module")
    def viewer_url(settings, outputs):
        return tier_url(outputs, settings, "viewer")


    def _upload(url, filename, content, mimetype):
        files = {"txtfile": (filename, content, mimetype)}
        return requests.post(url + "upload", files=files, timeout=10)


    def test_uploader_serves_its_form(uploader_url):
        response = requests.get(uploader_url, timeout=10)
        assert response.status_code == 200
        assert "Uploader" in response.text


    def test_viewer_serves_its_page(viewer_url):
        response = requests.get(viewer_url, timeout=10)
        assert response.status_code == 200
        assert "Viewer" in response.text
```

```
def test_text_file_uploaded_on_one_tier_appears_on_the_other(uploader_url, viewer_url):
    token = f"lab05-{uuid.uuid4()}"
    response = _upload(uploader_url, "lab05.txt", token.encode(), "text/plain")
    assert response.status_code == 200
    assert "Upload successful" in response.text

    def viewer_shows_token():
        return token in requests.get(viewer_url, timeout=10).text

    assert wait_until(viewer_shows_token, timeout=30, interval=2)


def test_non_txt_file_is_rejected(uploader_url):
    response = _upload(uploader_url, "picture.png", b"not text", "image/png")
    assert response.status_code == 415


def test_file_over_1mb_is_rejected(uploader_url):
    too_big = b"a" * (1024 * 1024 + 1)
    response = _upload(uploader_url, "big.txt", too_big, "text/plain")
    assert response.status_code == 413
```

## H4. Check recovery against the SLO

This is the Python test version of the Lab04 chaos experiment. It terminates a real instance and asserts that the endpoint is back within `RECOVERY_SLO_SECONDS`.

`tests/integration/test_recovery.py`

```
"""Terminate a real instance and measure how long recovery takes against your SLO.

Slow (a few minutes per tier). Run with:   pytest -m chaos -s
"""
import time

import pytest

from tests.helpers import probe, tier_url

pytestmark = pytest.mark.chaos


def _in_service_instance(aws, asg_name):
    group = aws.autoscaling.describe_auto_scaling_groups(
        AutoScalingGroupNames=[asg_name])["AutoScalingGroups"][0]
    for instance in group["Instances"]:
        if instance["LifecycleState"] == "InService":
            return instance["InstanceId"]
    return None


@pytest.mark.parametrize("tier_name", ["uploader", "viewer"])
def test_tier_recovers_within_the_slo(aws, settings, outputs, tier_name):
    tier = settings.tier(tier_name)
    url = tier_url(outputs, settings, tier_name)
    assert probe(url), "the endpoint must be healthy before the experiment starts"

    victim = _in_service_instance(aws, tier.asg_name)
    assert victim, f"no in-service instance in {tier.asg_name}"

    aws.autoscaling.terminate_instance_in_auto_scaling_group(
        InstanceId=victim, ShouldDecrementDesiredCapacity=False)
    started = time.time()

    saw_outage, recovered_after = False, None
    while time.time() - started < settings.recovery_slo_seconds * 2:
        if probe(url, timeout=2):
            if saw_outage:
                recovered_after = time.time() - started
                break
        else:
```

```
                saw_outage = True
            time.sleep(2)

        print(f"\n{tier_name}: terminated {victim}, recovery took {recovered_after}s "
              f"(SLO {settings.recovery_slo_seconds}s)")
        assert saw_outage, "the endpoint never went down, so there was nothing to recover from"
        assert recovered_after is not None, "the endpoint did not come back in time"
        assert recovered_after <= settings.recovery_slo_seconds
```

```
python3 -m py_compile tests/*.py tests/*/*.py
pytest --collect-only -m "integration or chaos"
```

Collection should list 14 integration tests and 2 chaos tests, and run none of them.

✅ **Checkpoint:** You can explain why `test_every_target_becomes_healthy` polls with `wait_until` instead of checking once.

---

## 14. Part I: Run It End to End

### Step 1. Build the stack

```
python create_stack.py
```

You should see the same sequence as the bash version, and then the two ALB URLs. The script also writes `stack_outputs.json` and `stack_outputs.env`.

### Step 2. Check the infrastructure and the app

```
pytest -m integration -v
```

`test_every_target_becomes_healthy` waits up to seven minutes, so the first run is slow. Re-run after that and it passes in seconds. All 14 should pass.

### Step 3. Measure recovery

```
pytest -m chaos -s
```

You will see one line per tier with the measured recovery time. Record both numbers.

### Step 4. Tighten the SLO and watch it fail

Change `RECOVERY_SLO_SECONDS` in `.env` to `60`, re-run `pytest -m chaos -s`, and read the failure message. Then set it back.

### Step 5. Prove the bash and Python versions agree

Open `stack_outputs.env` and compare it with the one `create_app_stack.sh` wrote in Lab04. The keys are identical and so are the subnet IDs. The ALB values differ because it is a new load balancer. This is why the next lab can read either file.

### Step 6. Clean up

```
python delete_stack.py
```

Run it a second time. It should finish without errors, because every delete step checks that something exists before deleting it. Then confirm nothing is left:

```
aws elbv2 describe-load-balancers --query "LoadBalancers[].LoadBalancerName"
aws autoscaling describe-auto-scaling-groups --query "AutoScalingGroups[].AutoScalingGroupName"
aws iam list-roles --query "Roles[?starts_with(RoleName, 'itmo-444-544')].RoleName"
```

To remove the load balancer and instances but keep the bucket, IAM roles and launch templates (to rebuild quickly), use `python`

`delete_stack.py --keep-foundation`.

## 15. Part J: Challenges

1. **Tags test.** Add an integration test that checks the launch template data tags every instance with `Name=uploader-app` or `Name=viewer-app`. Add a matching unit test against moto.
2. **A failing path.** Write a unit test where `Settings.from_values` receives a `RECOVERY_SLO_SECONDS` of `"fast"`. Decide what error a user should see, and change `config.py` so they see it.
3. **Dry run.** Add a `--dry-run` flag to `create_stack.py` that prints what each step would do without calling AWS. Which functions can stay unchanged, and which need a new parameter?
4. **Drift check.** Write an integration test that fails if someone adds a `0.0.0.0/0` rule on port 22 to the app security group. What would a daily run of this test give a team?
5. **Reflect.** Pick one place where the bash version was shorter than the Python version. Write two sentences on what the extra Python lines bought you.

## 16. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| `ModuleNotFoundError: No module named 'stack'` | Running from outside `lab05/`, or missing `stack/__init__.py` | `cd lab05`, and confirm the `__init__.py` files exist (Part 0.3) |
| `ModuleNotFoundError: boto3` or `moto` | Virtual environment not active | `source .venv/bin/activate`, then `pip install -r requirements.txt` |
| `ConfigError: Missing keys in .env: ...` | `.env` is missing keys from earlier labs | Add the named keys. Copy values from your Lab04 `.env` |
| `pytest` collects 0 tests | Running from the wrong folder, or `pytest.ini` missing | Run from `lab05/` and check `pytest.ini` exists |
| `pytest -m integration` says all deselected | `addopts` ordering on an older pytest | Use `pytest -o addopts="" -m integration` |
| Unit tests fail with `NoCredentialsError` | A very old `moto` that does not set fake credentials | Upgrade with `pip install -U "moto[ec2,elbv2,autoscaling,iam,s3]"` |
| `BucketAlreadyExists` or a 403 on `head_bucket` | Another account owns that bucket name | Pick a unique `BUCKET_NAME` in `.env` (include your NetID) |
| `InvalidParameterValue: ... iamInstanceProfile` right after create | IAM had not propagated | Re-run `python create_stack.py`. It is idempotent. Increase `iam_wait` if it repeats |
| `StackError: The default VPC has no subnet in ...` | Your region lacks that AZ in the default VPC | Pick two AZs that exist and set `AZ_A` and `AZ_B` |
| `test_every_target_becomes_healthy` times out | Instances still running `npm install`, or the app crashed | Wait, then check `systemctl status uploader.service` over SSH |
| `AccessDenied` on `simulate_principal_policy` | Your IAM user cannot call `iam:SimulatePrincipalPolicy` | Add that permission to your user, or skip those two tests with `-k "not role"` |
| `test_text_file_uploaded...` fails with a 502 | ALB reached an instance that was mid-restart | Wait a minute and re-run |
| `delete_stack.py` waits a long time on the security group | The ALB's network interfaces are still releasing | It retries for about 2.5 minutes. Wait, or re-run |
| Chaos test: "the endpoint never went down" | Termination was not noticed in the test window | Re-run. Check the ASG replaced the instance in the console |

## 17. Submission Checklist

- ☐ Output of `ls -a lab05` (or a screenshot) showing the layout from Part 0 and section 4.
- ☐ Screenshot of `pytest` showing 19 passed.

- [ ] Screenshot of one "break it on purpose" experiment: the changed line and the failing test message.
- [ ] Screenshot of `python create_stack.py` completing, with both URLs.
- [ ] Screenshot of `pytest -m integration -v` with all 14 tests passing.
- [ ] Output of `pytest -m chaos -s` for both tiers, and the same run failing after you tightened the SLO.
- [ ] Screenshot of `python delete_stack.py` completing, and a second run completing with no errors.
- [ ] One paragraph on what moto proved, what it could not, and which tests cover the gap.
- [ ] One paragraph comparing `ignore_errors("EntityAlreadyExists")` with `2>/dev/null || true`.
- [ ] Your answer to Challenge 5.

---

## 18. Key Terms Glossary

- **boto3:** The AWS SDK for Python. The AWS CLI is built on the same underlying library.
- **botocore:** The low-level library under both boto3 and the CLI. It defines the exception `ClientError`.
- **Client:** A boto3 object for one AWS service (for example `ec2` or `s3`), created for one region.
- **Response:** The dictionary a boto3 call returns. You index into it instead of using `--query`.
- **Waiter:** A built-in helper that polls until a resource reaches a state, such as an ALB becoming active.
- **Paginator:** A helper that follows `NextToken` across pages of results so you do not have to.
- **ClientError:** The exception boto3 raises for AWS-side errors. The specific problem is in `err.response["Error"]["Code"]`.
- **Idempotent:** Safe to run more than once with the same result. Every `ensure_*` function is idempotent.
- **Dataclass:** A Python class that stores named fields and generates its constructor and comparison.
- **Pure function:** A function whose output depends only on its inputs and which changes nothing outside itself. Easy to test.
- **Unit test:** A fast test of one piece of code in isolation.
- **Integration test:** A test that checks several real components working together, here your deployed AWS stack.
- **Chaos test:** A test that causes a failure on purpose and checks the system recovers.
- **Fixture:** A pytest function that prepares something (settings, a fake AWS account) and hands it to tests that ask for it.
- **Marker:** A pytest label such as `integration` that lets you choose which tests to run.
- **moto:** A library that fakes AWS in memory for tests. It cannot run instances or serve HTTP.
- **Least privilege:** Granting only the permissions a task needs. `simulate_principal_policy` lets you test it.
- **SLO:** A measurable target. Here, recovery within `RECOVERY_SLO_SECONDS`.

---

## 19. References


- boto3 documentation: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html
- boto3 error handling: https://boto3.amazonaws.com/v1/documentation/api/latest/guide/error-handling.html
- pytest documentation: https://docs.pytest.org/
- moto documentation: https://docs.getmoto.org/
- IAM policy simulator API: https://docs.aws.amazon.com/IAM/latest/APIReference/API_SimulatePrincipalPolicy.html